



DR.D.Y.PATIL INSTITUTE OF TECHNOLOGY

PIMPRI, PUNE

(Affiliated to Savitribai Phule Pune University)

A PROJECT REPORT ON

AI for Sustainable Urban Drainage System for Effective Waterlogging Prediction and Management

SUBMITTED TOWARDS THE PARTIAL
FULFILLMENT OF THE REQUIREMENTS OF

**BACHELOR OF ENGINEERING (Artificial Intelligence and
Data Science)**

BY

Tushar Chavhan

Exam No: B400240944

Sakshi Ingale

Exam No: B400240977

Vrushali Dhage

Exam No: B400240953

Snehal Godage

Exam No: B400240969

Under The Guidance of

Prof. Shubhangi Bhawari



Department of Artificial Intelligence and Data Science

DR.D.Y.PATIL INSTITUTE OF TECHNOLOGY

A.Y 2025-2026



DR.D.Y.PATIL INSTITUTE OF TECHNOLOGY
Department of Artificial Intelligence and Data Science

CERTIFICATE

This is to certify that the Project Entitled

**AI for Sustainable Urban Drainage System for Effective
Waterlogging Prediction and Management**

Submitted by

Tushar Chavhan	Exam No: B400240944
Sakshi Ingale	Exam No: B400240977
Vrushali Dhage	Exam No: B400240953
Snehal Godage	Exam No: B400240969

is a bonafide work carried out by students under the supervision of Prof. Shubhangi Bhawari and it is submitted towards the partial fulfillment of the requirement of Bachelor of Engineering (Artificial Intelligence and Data Science).

Prof. Shubhangi Bhawari
Internal Guide
Dept. of AI and DS

Dr. Shubhangi Vairagar
H.O.D
Dept. of AI and DS

Dr. Nitin P. Sherje
Principal
DIT, Pimpri

Signature of Project Coordinator

Signature of External Examiner

PROJECT APPROVAL SHEET

A Project Title

AI for Sustainable Urban Drainage System for Effective Waterlogging Prediction
and Management

Is successfully completed by

Tushar Chavhan	Exam No: B400240944
Sakshi Ingale	Exam No: B400240977
Vrushali Dhage	Exam No: B400240953
Snehal Godage	Exam No: B400240969

at

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

(Dr.D.Y.Patil Institute of Technology).

SAVITRIBAI PHULE PUNE UNIVERSITY,PUNE

ACADEMIC YEAR 2025-26

Prof. Shubhangi Bhawari
Internal Guide
Dept. of AI and DS

Dr. Shubhangi Vairagar
H.O.D
Dept. of AI and DS

Abstract

A multi-source Artificial Intelligence (AI) framework for enhancing and maintaining urban waterlogging prediction is presented in this paper. It accomplishes this by combining actual weather data, drainage maps created using a Geographic Information System (GIS), and Internet of Things (IoT) water level sensors to deliver precise and up-to-date flood insights. The growing problems of urban flooding, which are made worse by climate change, fast urbanization, and poor drainage infrastructure, are the driving force behind this study. To create a thorough understanding of drainage behavior, the suggested framework integrates information from IoT-based water-level networks, spatial drainage models, and local weather station data. Using artificial intelligence (AI) methods like ensemble learning and Long Short-Term Memory (LSTM) neural networks, the system analyzes both live and historical data to make highly accurate predictions about possible waterlogging incidents. The integration of various data sources, the dynamic updating of risk profiles through real-time sensor feedback, and the combination of data-driven AI models and hydrodynamic simulations are what make it novel. The framework's capacity to improve proactive drainage management and facilitate prompt emergency response is demonstrated by the outcomes of pilot testing and validation using actual flood data. Overall, this framework advances the development of intelligent and climate-resilient urban drainage systems, supporting the more general objectives of adaptive and sustainable urban development.

Acknowledgments

*It gives us great pleasure in presenting the preliminary project report on ‘**AI for Sustainable Urban Drainage System for Effective Waterlogging Prediction and Management**’.*

*I would like to take this opportunity to thank my internal guide **Prof. Shubhangi Bawari** for giving me all the help and guidance I needed. I am really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*I am also grateful to **Prof. Shubhangi Vairagar**, Head of the Department of Artificial Intelligence and Data Science, Dr. D. Y. Patil Institute of Technology, Pimpri for her valuable support and suggestions.*

*In the end our special thanks to **Dr. Nitin Sherje** for providing various resources such as laboratory with all needed software platforms, continuous Internet connection, for Our Project.*

Tushar Chavhan
Sakshi Ingale
Vrushali Dhage
Snehal Godage
(B.E. AI and DS)

Contents

1	Synopsis	2
1.1	Project Title	2
1.2	Project Option	2
1.3	Internal Guide	2
1.4	Sponsorship and External Guide	2
1.5	Technical Keywords (As per ACM Keywords)	2
1.6	Problem Statement	3
1.7	Abstract	3
1.8	Goals and Objectives	4
1.9	Relevant Mathematics Associated with the Project	4
1.10	Names of Conferences / Journals where papers can be published . .	6
1.11	Review of Conference/Journal Papers supporting Project idea . . .	6
1.12	Plan of Project Execution	8
2	Technical Keywords	10
2.1	Area of Project	10
2.2	Technical Keywords	10
3	Introduction	12
3.1	Project Idea	12
3.2	Motivation of the Project	12
3.3	Literature Survey	13
4	Problem Definition and scope	14

4.1	Problem Statement	14
4.1.1	Goals and Objectives	14
4.1.2	Statement of Scope	15
4.2	Major Constraints	15
4.3	Methodologies of Problem Solving and Efficiency Issues	16
4.4	Outcome	16
4.5	Applications	17
4.6	Hardware Resources Required	18
4.7	Software Resources Required	18
5	Project Plan	20
5.1	Project Estimates	20
5.1.1	Reconciled Estimates	20
5.1.2	Project Resources	21
5.2	Risk Management w.r.t. NP Hard analysis	21
5.2.1	Risk Identification	21
5.2.2	Risk Analysis	22
5.2.3	Overview of Risk Mitigation, Monitoring, Management . . .	22
5.3	Project Schedule	24
5.3.1	Project task set	24
5.3.2	Task network	25
5.3.3	Timeline Chart	26
5.4	Team Organization	26
5.4.1	Team Structure	26
5.4.2	Management Reporting and Communication	27
6	Software requirement specification	28
6.1	Introduction	28
6.1.1	Purpose and Scope of Document	28
6.1.2	Overview of Responsibilities of Developer	28

6.2	Usage Scenario	29
6.2.1	User Profiles	29
6.2.2	Use-cases	29
6.2.3	Use Case View	31
6.3	Data Model and Description	31
6.3.1	Data Description	31
6.3.2	Data Objects and Relationships	32
6.4	Functional Model and Description	33
6.4.1	Data Flow Diagram	33
6.4.2	Activity Diagram	34
6.4.3	Non Functional Requirements	35
6.4.4	State Diagram	36
6.4.5	Design Constraints	37
6.4.6	Software Interface Description	38
7	Detailed Design Document using Appendix A and B	39
7.1	Introduction	39
7.2	Architectural Design	39
7.3	Data Design (using Appendices A and B)	40
7.3.1	Internal Software Data Structure	40
7.3.2	Global Data Structure	41
7.3.3	Temporary Data Structure	41
7.3.4	Database Description	41
7.4	Component Design	42
7.4.1	Class Diagram	42
7.4.2	Interaction Diagram	43
7.4.3	Algorithms	44
8	Project Implementation	45
8.1	Introduction	45

8.2	Tools and Technologies Used	45
8.3	Methodologies/Algorithm Details	45
8.3.1	Algorithm 1: Waterlogging Prediction	46
8.3.2	Algorithm 2: Alert Generation	46
8.4	Verification and Validation for Acceptance	47
9	Software Testing	48
9.1	Type of Testing Used	48
9.2	Test Cases and Test Results	48
9.2.1	Unit Testing	49
9.2.2	Integration Testing	49
9.2.3	System Testing	49
10	Results	50
10.1	Screenshots	50
10.2	Outputs	52
11	Deployment and Maintenance	53
11.1	Installation and Un-installation	53
11.2	User Help	54
12	Conclusion and Future Scope	55
12.1	Summary	55
12.2	Conclusion	55
12.3	Future Scope	55
A	References	57
B	Laboratory Assignments on Project Analysis of Algorithmic Design	59
C	Laboratory Assignments on Project Quality and Reliability Testing of Project Design	61

D Project Planner	63
E Published Papers	65
F Plagiarism Report	66
G Term-II Project Laboratory Assignments	67
H Information of Project Group Members	70

List of Figures

6.1	Use Case Diagram of Smart Drainage System	31
6.2	Level 0 Data Flow Diagram	33
6.3	Level 1 Data Flow Diagram	34
6.4	Activity Diagram	35
6.5	State Transition Diagram	37
7.1	Architecture Diagram	40
7.2	Class Diagram	43
10.1	Dashboard	50
10.2	Graphs and Charts	51
10.3	Live Data Entry	51
10.4	Searching map locations	51
10.5	Searching map locations	52
10.6	Chatbot interface for user interaction	52
F.1	Plagiarism Report 1	66
F.2	Plagiarism Report 2	66
F.3	Plagiarism Report 3	66

List of Tables

4.1	Hardware Requirements	18
5.1	Short Risk Analysis Table	22
5.2	Risk Probability definitions	22
5.3	Risk Impact definitions	22
5.4	Risk Detail Description	23
5.5	Risk Detail Description (Dataset Related)	23
5.6	Risk Detail Description (Technology Related)	24
5.7	Task Network – Task Dependencies	25
5.8	Project Timeline Chart	26
5.9	Team Structure and Roles	27
6.1	Use Case Table	30
9.1	Unit Testing	49
9.2	Integration Testing	49
9.3	System Testing	49
C.1	Black Box Testing	62

1. Synopsis

1.1 Project Title

AI for Sustainable Urban Drainage System for Effective Waterlogging Prediction and Management

1.2 Project Option

Internal Project

1.3 Internal Guide

Prof. Shubhangi Bhawari

1.4 Sponsorship and External Guide

No.

1.5 Technical Keywords (As per ACM Keywords)

1. Computer Systems Organization

- C.2 Computer-Communication Networks
 - C.2.4 Distributed Systems
 - * Client-server architecture
 - * Distributed applications

* Reliability and fault tolerance

2. Artificial Intelligence

- I.2 Artificial Intelligence
 - I.2.1 Applications and Expert Systems
 - I.2.6 Machine Learning

3. Smart City Systems

- J.7 Computers in Other Systems
 - Urban Infrastructure
 - Smart Drainage and Water Management

4. Information Systems

- H.4 Information Systems Applications
 - Decision Support Systems
 - Real Time Monitoring Systems

1.6 Problem Statement

Urban areas frequently face severe waterlogging issues due to inefficient drainage systems, rapid urbanization, and unpredictable rainfall patterns. Traditional drainage monitoring systems lack real-time analysis and predictive capabilities. The problem is to design and develop an intelligent system that can monitor drainage conditions, analyze multiple environmental parameters, and predict flood risks in real-time to assist authorities in effective water management and decision-making.

1.7 Abstract

This project presents an AI-based Smart Urban Drainage System designed to predict and manage waterlogging in urban areas. The system integrates simulated IoT sensor data, environmental parameters such as rainfall and water level, and machine learning techniques to analyze flood risks. A hybrid machine learning approach combining deep learning (CNN-LSTM) and ensemble models (XGBoost) is used for predictive analysis. The system provides real-time monitoring through a web-based dashboard, visualizes drainage conditions using maps, and generates alerts based on risk levels. Additionally, it offers decision support by suggesting preventive actions. Due to the absence of real-time sensor deployment, synthetic

data is used for simulation. The proposed system aims to enhance urban flood management and contribute to smart city infrastructure.

1.8 Goals and Objectives

- To design a smart drainage monitoring system
- To analyze multi-source data such as rainfall, water level, and blockage
- To predict flood risk using AI/ML models
- To provide real-time alerts and recommendations
- To visualize data using dashboards and maps
- To simulate an IoT-based environment

1.9 Relevant Mathematics Associated with the Project

System Description:

- **Input:**
 - Rainfall intensity (mm/hr)
 - Water level (m)
 - Drain capacity (%)
 - Blockage factor
- **Output:**
 - Flood risk level (Low / Medium / High)
 - Probability score
 - Alerts and recommendations
- **Data Structures and Processing Techniques:**
 - **Arrays for Time-Series Data:** Arrays are used to represent time-series data such as rainfall intensity, water level measurements, and sensor readings over time. These structures enable efficient sequential data processing and are particularly useful for feeding input into machine learning models that require temporal data patterns.
 - **Dictionaries (JSON) for API Communication:** Dictionaries are used in the form of JSON (JavaScript Object Notation) for communication between the frontend and backend. They facilitate

structured data exchange in APIs, allowing efficient transmission of parameters such as water level, rainfall, and prediction results.

- **DataFrames for Dataset Handling:** DataFrames are used for handling and processing datasets during the training and evaluation of machine learning models. They provide a tabular structure that supports efficient data manipulation, filtering, and preprocessing operations.

- **Functional Representation:**

The relationship between input parameters and flood risk can be mathematically represented as:

$$R = f(x_1, x_2, x_3, x_4)$$

where:

- R represents the predicted flood risk level
- x_1 represents rainfall intensity
- x_2 represents water level
- x_3 represents drainage capacity
- x_4 represents blockage factor

Thus, the function $f(\cdot)$ models the mapping between environmental parameters and the corresponding flood risk.

- **Mathematical Formulation (if applicable):**

The prediction model can be represented using Logistic Regression as:

$$P(y = 1) = \frac{1}{1 + e^{-(w_1x_1 + w_2x_2 + \dots + b)}}$$

where $x_1, x_2, \dots$ are input features such as rainfall, water level, and drainage capacity, and $w_1, w_2, \dots$ are corresponding weights.

- **Success Conditions:**

- Accurate risk prediction
- Reliable alert generation
- Efficient system performance

Real-time system response

Proper alert generation

- **Failure Conditions:**

- Incorrect prediction due to noisy data
- Missing sensor inputs
- System latency

1.10 Names of Conferences / Journals where papers can be published

- IEEE International Conference on Smart Cities
- ACM SIGKDD Conference
- Conferences in IIT
- Savitribai Phule Pune University Conferences

1.11 Review of Conference/Journal Papers supporting Project idea

1. D. Misra, G. Das, and D. Das, “Artificial intelligent based smart drainage system,” 2022, preprint available at Research Square. [Online]. Available: <https://doi.org/10.21203/rs.3.rs-1338435/v1>
2. R. Dabas, T. Imam, F. Safwat, S. Rizwan, and K. Alam, “Smart drainage system for urban flood prevention.”
3. M. M. Saddiqi, W. Zhao, S. Cotterill, and R. K. Dereli, “Smart management of combined sewer overflows: From an ancient technology to artificial intelligence,” *Wiley Interdisciplinary Reviews: Water*, vol. 10, no. 3, p. e1635, 2023. [Online]. Available: <https://doi.org/10.1002/wat2.1635>
4. S. Boughandjioua, F. Laouacheria, and N. Azizi, “Integrating artificial intelligence for urban drainage systems aided decision: State of the art,” *International Journal of Artificial Intelligence: Theory and Applications*, vol. 2, no. 2, pp. 1–9, 2023. [Online]. Available: <https://int-j-aita.univ-annaba.dz/wp-content/themes/cbusiness-investment/view/vol2iss2/vol2iss2-1.pdf>
5. D. P. Selvam, G. G. Reddy, and N. M. Kumar, “Designing a smart and safe drainage system using artificial intelligence,” *International Journal of Engineering Research and Technology (IJERT)*, vol. 9, pp. 497–500, 2020. [Online]. Available: <https://www.ijert.org/research/designing-a-smart-and-safe-drainage-system-using-artificial-intelligence-IJERT.pdf>
6. K. Haripriya, S. S. Hammed, C. Preethi, S. Pavalarajan, R. Padmakumar, and M. M. Surya, “Automated drainage management system: AI and IoT-based

- blockage detection for smarter cities,” in *7th International Conference on Inventive Material Science and Applications (ICIMA)*, IEEE, 2025, pp. 998–1004. [Online]. Available: <https://doi.org/10.1109/ICIMA64861.2025.11073840>
7. S. H. Kwon and J. H. Kim, “Machine learning and urban drainage systems: State-of-the-art review,” *Water*, vol. 13, no. 24, p. 3545, 2021. [Online]. Available: <https://doi.org/10.3390/w13243545>
 8. P. S. Thanigaivelu, A. Sairam, P. Vijayan, A. S. Kumar, N. Mohankumar, and T. R. GaneshBabu, “Revolutionizing urban drainage: A smart IoT approach to stormwater management using adaboosting algorithm,” in *International Conference on Advances in Modern Age Technologies for Health and Engineering Science (AMATHE)*, IEEE, 2024, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/AMATHE61652.2024.10582217>
 9. R. Sekita, T. Watanabe, T. Yoshizawa, and K. Nakanishi, “Reliability improvement of rainwater-drainage system using IoT and AI,” in *Annual Reliability and Maintainability Symposium (RAMS)*, IEEE, 2024, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/RAMS51492.2024.10457666>
 10. S. Veerappan, “Edge-enabled smart storm water drainage systems: A real-time analytics framework for urban flood management,” *Journal of Smart Infrastructure and Environmental Sustainability*, vol. 1, no. 1, pp. 52–59, 2024. [Online]. Available: <https://doi.org/10.17051/JSIES/01.01.06>
 11. U. K. Sharma and A. Maurya, “AI for predictive urban water drainage systems,” *International Journal of Progressive Research in Engineering Management and Science*, vol. 4, no. 12, pp. 763–767, 2024. [Online]. Available: <https://www.doi.org/10.58257/IJPREMS37638>
 12. P.-C. Huang and K. T. Lee, “An alternative for predicting real-time water levels of urban drainage systems,” *Journal of Environmental Management*, vol. 347, p. 119099, 2023. [Online]. Available: <https://doi.org/10.1016/j.jenvman.2023.119099>
 13. Y. Liu, Y. Liu, J. Zheng, F. Chai, and H. Ren, “Intelligent prediction method for waterlogging risk based on AI and numerical model,” *Water*, vol. 14, no. 15, p. 2282, 2022. [Online]. Available: <https://doi.org/10.3390/w14152282>

1.12 Plan of Project Execution

The project is executed in multiple phases, each focusing on a specific stage of development.

1. Phase 1: Problem Definition and Literature Survey

In this phase, the problem of urban waterlogging and inefficient drainage systems is identified. A detailed literature survey is conducted to study existing solutions, AI-based prediction models, and smart drainage systems. This helps in understanding the research gap and defining the objectives of the project.

2. Phase 2: Dataset Collection and Preprocessing

Relevant datasets related to rainfall, water levels, and drainage capacity are collected. The data is then preprocessed by handling missing values, normalization, and feature selection to make it suitable for machine learning model training.

3. Phase 3: Machine Learning Model Development (CNN-LSTM + XGBoost)

A hybrid machine learning model is developed using CNN-LSTM for feature extraction and XGBoost for classification. The model is trained on the processed dataset to predict flood risk levels such as low, medium, and high.

4. Phase 4: Backend Development (FastAPI APIs)

The backend is developed using FastAPI to handle data processing, model inference, and API communication. It provides endpoints for prediction, alerts, and system interaction.

5. Phase 5: Frontend Development (React Dashboard)

A user-friendly dashboard is developed using React. It visualizes real-time data, prediction results, alerts, and system status in an interactive and intuitive manner.

6. Phase 6: System Integration (ML + Backend + Frontend)

In this phase, the machine learning model is integrated with the backend APIs, and the frontend is connected to the backend. This enables seamless data flow and real-time prediction functionality.

7. Phase 7: Testing and Validation

The system is tested for accuracy, performance, and reliability. The machine learning model is evaluated using appropriate metrics, and the overall system is validated to ensure correct predictions and smooth operation.

8. Phase 8: Documentation and Report Preparation

All project details, including design, implementation, results, and conclusions, are documented. A comprehensive report is prepared to describe the complete system and its outcomes.

2. Technical Keywords

2.1 Area of Project

The project falls under the domain of Artificial Intelligence and Smart City Systems. It integrates concepts from Machine Learning, Internet of Things (IoT), and Web Technologies to develop an intelligent drainage monitoring and flood prediction system.

Primary Areas of the Project:

- Artificial Intelligence and Machine Learning
- Smart City Infrastructure
- Environmental Monitoring Systems
- Internet of Things (IoT)
- Web-based Decision Support Systems

The system is designed to simulate real-time drainage monitoring and provide predictive insights for urban water management.

2.2 Technical Keywords

1. C. Computer Systems Organization

(a) C.2 Computer-Communication Networks

i. C.2.4 Distributed Systems

- A. Client/server architecture
- B. Distributed applications
- C. Distributed databases
- D. Network operating systems
- E. Distributed file systems

F. Security and reliability in distributed applications

2. I. Computing Methodologies

(a) I.2 Artificial Intelligence

- i. Machine Learning
- ii. Neural Networks (CNN-LSTM)
- iii. Ensemble Learning (XGBoost)

3. H. Information Systems

(a) H.4 Information Systems Applications

- i. Decision Support Systems
- ii. Real-time Monitoring Systems

4. J. Computer Applications

(a) J.7 Computers in Other Systems

- i. Smart Cities
- ii. Urban Infrastructure Management
- iii. Sustainable Drainage Systems

3. Introduction

3.1 Project Idea

The proposed project aims to develop an intelligent system for urban drainage monitoring and waterlogging prediction using Artificial Intelligence. Rapid urbanization and climate change have significantly increased the frequency of urban flooding, making traditional drainage systems insufficient for effective water management. The core idea of this project is to integrate multiple data sources such as rainfall intensity, water level, and drainage capacity into a unified system that can analyze and predict flood risks in real-time. The system uses machine learning techniques, including hybrid models combining deep learning (CNN-LSTM) and ensemble methods (XGBoost), to generate accurate predictions. A web-based platform is developed to visualize drainage conditions through dashboards and maps, providing authorities with actionable insights and recommendations. Since real IoT deployment is not feasible in the current setup, the system simulates sensor data to replicate real-world scenarios.

3.2 Motivation of the Project

Urban flooding is a major issue affecting infrastructure, transportation, and public safety. Existing drainage systems lack predictive capabilities and operate reactively rather than proactively. This motivated the development of a smart system that can anticipate flood conditions and assist in decision-making.

The key motivations behind this project are:

- Increasing urban flooding due to climate change and poor drainage infrastructure
- Lack of real-time monitoring and predictive analysis in existing systems
- Need for intelligent decision support systems in smart cities

- Advancement in Artificial Intelligence and IoT technologies
- Requirement for cost-effective and scalable solutions

The project aims to bridge the gap between traditional drainage systems and modern smart city solutions by introducing AI-driven prediction and monitoring capabilities.

3.3 Literature Survey

- Several studies have explored the use of AI, IoT, and GIS for urban flood prediction and drainage management. LSTM-based models have shown high accuracy in forecasting water levels by capturing temporal dependencies in rainfall data. Ensemble learning approaches, combining multiple models, have been effective in reducing false alarms and improving reliability. GIS integration enables spatial mapping of flood-prone zones, while IoT sensors provide real-time feedback on drainage conditions.
- For instance, *P. Huang and K. T. Lee (2023)* proposed a real-time water level prediction model using LSTM networks that outperformed traditional hydraulic models. Similarly, *R. Dabas et al. (2025)* introduced a smart drainage system using AI and IoT for proactive flood prevention. Studies by *Kwon and Kim (2021)* and *Liu et al. (2022)* emphasized the importance of combining numerical simulations with AI for more reliable flood predictions.
- Mathematically, these systems rely on spatiotemporal modeling where rainfall intensity (R_t), water-level readings (W_t), and geographic features ($G_{x,y}$) are inputs to predict the probability of flooding:

$$P_{flood}(x, y, t) = f(R_t, W_t, G_{x,y})$$

Deep learning architectures such as LSTM can be represented as:

$$h_t = \sigma(W_h h_{t-1} + W_x X_t + b), \quad \hat{Y}_t = \phi(W_y h_t + c)$$

where h_t is the hidden state, X_t the input vector, and $\hat{Y}_t$ the predicted flood risk score.

- From the reviewed works, it is evident that integrating AI, IoT, and GIS data into a unified system provides superior accuracy and efficiency compared to traditional hydrological models. This literature strongly supports the proposed project's objective of developing a **multi-source AI-driven framework** for sustainable and real-time urban drainage management.

4. Problem Definition and scope

4.1 Problem Statement

Urban areas frequently experience waterlogging due to inefficient drainage systems, increasing rainfall intensity, and rapid urbanization. Traditional drainage systems lack real-time monitoring and predictive capabilities, leading to delayed responses and severe consequences such as traffic disruption, infrastructure damage, and public safety risks. The problem is to design and develop an intelligent system that can monitor environmental parameters, analyze drainage conditions, and predict waterlogging risks in advance. The system should provide real-time alerts and decision support to assist authorities in managing urban flooding effectively.

4.1.1 Goals and Objectives

Goal:

To develop an AI-based smart drainage system capable of predicting waterlogging and providing real-time monitoring and decision support.

Objectives:

- To collect and process input data such as rainfall, water level, and drainage capacity
 - To implement machine learning models for flood risk prediction
 - To design a real-time dashboard for visualization of drainage conditions
 - To generate alerts and recommendations based on predicted risk levels
 - To simulate IoT sensor data for system testing
 - To integrate backend (FastAPI), frontend (React), and machine learning models
-
- Overall goals and objectives of the software, including input and output description with necessary syntax and format, are defined.

4.1.2 Statement of Scope

- The system focuses on predicting waterlogging using environmental parameters such as rainfall, water level, and drainage capacity.
- It includes data processing, machine learning-based prediction, alert generation, and visualization through a dashboard.
- Input constraints include valid numerical ranges for rainfall, water level, and capacity, along with proper validation of input data.
- The system provides outputs in the form of risk levels (Low, Medium, High), probability scores, and alert messages.
- The scope defines what the system will do, such as prediction and monitoring, and excludes aspects like physical drainage infrastructure modification.

4.2 Major Constraints

- **Data Constraints**
 - Lack of real-time IoT sensor data
 - Use of simulated or synthetic datasets
- **Computational Constraints**
 - Limited resources for running deep learning models
 - Need to simplify machine learning models for real-time use
- **Hardware Constraints**
 - No physical sensor deployment
 - No embedded system integration
- **Time Constraints**
 - Limited development time
 - Need for simplified implementation
- **Accuracy Constraints**
 - Predictions depend on data quality
 - Simulated data may reduce real-world accuracy
- **Integration Constraints**
 - Combining machine learning, backend, and frontend systems
 - Ensuring smooth API communication

4.3 Methodologies of Problem Solving and Efficiency Issues

- The problem of urban waterlogging prediction can be approached using several methodologies. Traditional hydrological and hydraulic simulations rely on physical equations to model water flow and drainage, but they are computationally expensive and less adaptable to real-time changes. In contrast, **Artificial Intelligence (AI)** and **Machine Learning (ML)** methods provide data-driven alternatives that can learn from historical and real-time data for rapid, adaptive forecasting.
- The proposed approach uses a combination of **Long Short-Term Memory (LSTM)** networks and **Ensemble Learning** to capture both temporal and spatial variations in water levels. LSTM networks are highly effective for time-series prediction, while ensemble models combine multiple predictors to reduce false alarms and enhance accuracy.
- The system also incorporates **IoT sensor feedback** and **GIS data fusion** to improve contextual understanding of drainage behavior. Efficiency is ensured by deploying the model on **edge devices** for local processing, minimizing latency and communication overhead.
- Performance parameters considered for evaluating efficiency include:
 - **Prediction Accuracy (R^2)** – measures how closely predicted water levels match observed data.
 - **Mean Relative Error (MRE)** – ensures precision within acceptable error bounds.
 - **Computation Time** – optimized by model quantization and edge-based inference.
 - **System Latency** – time taken from data acquisition to alert generation.
- Compared to traditional SCADA and cloud-only systems, the proposed hybrid edge–cloud architecture achieves up to **82% faster response time** and maintains prediction accuracy above **90%**, ensuring real-time flood awareness and efficient resource management.

4.4 Outcome

- The outcome of this project is an **AI-driven sustainable urban drainage management system** capable of accurately predicting and managing waterlogging in real time.

- The framework integrates IoT-based sensors, GIS data, and meteorological inputs to provide early warnings and visual dashboards for decision support.
- The implemented model achieves high predictive accuracy with minimal computational load and demonstrates the feasibility of edge-based flood prediction systems for smart cities.
- Ultimately, the system enhances urban resilience, reduces flood-related damage, and supports the vision of intelligent, sustainable urban infrastructure.

4.5 Applications

- **Smart City Infrastructure:** Real-time flood prediction and management systems for municipal corporations and urban planners.
- **Disaster Management Authorities:** Early warning systems to plan and coordinate evacuation or emergency responses.
- **Drainage and Water Resource Departments:** Monitoring and optimizing drainage networks to prevent overflow and blockages.
- **Environmental Monitoring:** Continuous tracking of rainfall patterns, runoff behavior, and soil saturation for climate studies.
- **Infrastructure Planning:** Assisting engineers and architects in designing resilient drainage layouts for new urban developments.

4.6 Hardware Resources Required

Sr. No.	Parameter	Minimum Requirement	Justification
1	CPU Speed	2.0 GHz (Quad-Core or higher)	Required for efficient execution of AI model training and real-time data processing.
2	RAM	8 GB	To handle large datasets (GIS, IoT readings, weather data) and model operations smoothly.
3	Storage	256 GB SSD	For faster data access and storage of model checkpoints and datasets.
4	GPU (Optional)	NVIDIA GPU with CUDA support	Accelerates training of deep learning models (LSTM/GRU).
5	Network Interface	Wi-Fi / Ethernet / LoRa Module	Needed for continuous data transfer between IoT devices and the server.
6	Microcontroller Nodes	ESP32 / Raspberry Pi / Arduino UNO	Used for IoT-based water-level sensing and data transmission.
7	Sensors	Ultrasonic Sensor, Rain Gauge, Flow Sensor	For real-time monitoring of rain-fall, water depth, and drainage flow rate.

Table 4.1: Hardware Requirements

4.7 Software Resources Required

Platform: AI-driven predictive modeling and IoT integration environment.

1. **Operating System:** Windows 10 / Ubuntu 22.04 LTS (Ubuntu preferred for Python, TensorFlow, and IoT integration)
2. **IDE:** Visual Studio Code, Jupyter Notebook, or PyCharm (for Python coding, data preprocessing, and ML model development)
3. **Programming Languages:** Python (primary language for AI, ML, and IoT modules) C / C++ (for embedded programming of IoT sensors)

4. **Libraries and Frameworks:** TensorFlow, Keras, Scikit-learn, Pandas, NumPy, Matplotlib, Flask / FastAPI (for backend integration), and GIS libraries (e.g., GeoPandas, Folium)
5. **Database:** PostgreSQL with PostGIS extension or MongoDB (to store sensor data, model outputs, and spatial maps)
6. **Visualization Tools:** Power BI, Grafana, or Dash for real-time monitoring and visual analytics.

5. Project Plan

5.1 Project Estimates

The project is developed using the Waterfall Model, where each phase is completed sequentially, including requirement analysis, design, implementation, testing, and deployment. Estimates are derived based on prior assignments and project planning activities.

5.1.1 Reconciled Estimates

5.1.1.1 Cost Estimate

Since this is an academic internal project, no direct financial cost is involved. However, an estimated cost in terms of resources is as follows:

- Development tools: Open-source (no cost)
- Hardware: Personal computer/laptop
- Internet and electricity usage

Estimated Cost: Minimal (academic project environment)

5.1.1.2 Time Estimates

The estimated time required for each phase of the project is as follows:

1. Problem Definition and Literature Survey – 2 weeks
2. Dataset Collection and Preprocessing – 3 weeks
3. Machine Learning Model Development – 4 weeks
4. Backend Development – 3 weeks

5. Frontend Development – 3 weeks
6. System Integration – 2 weeks
7. Testing and Validation – 2 weeks
8. Documentation – 2 weeks

5.1.2 Project Resources

- **People**
 - 3–4 Students (Development team)
 - 1 Internal Guide (Project supervision)
- **Hardware**
 - Laptop/Desktop with minimum 2 GHz CPU and 3 GB RAM
- **Software**
 - Visual Studio Code
 - Jupyter Notebook
 - MySQL
- **Tools & Technologies**
 - Python (Backend & Machine Learning)
 - JavaScript (Frontend – React)
 - FastAPI (Backend framework)
 - TensorFlow and XGBoost (ML models)

5.2 Risk Management w.r.t. NP Hard analysis

The project involves managing uncertainties related to data availability, system performance, and model accuracy. Although the problem is not strictly NP-hard, it involves complex decision-making due to multiple influencing factors such as rainfall, water level, and drainage capacity.

5.2.1 Risk Identification

For risks identification, review of scope document, requirements specifications and schedule is done. Answers to questionnaire revealed some risks. Each risk is categorized as per the categories mentioned in the standard risk framework.

Please refer tables 5.4, 5.5 and 5.6 for all the risks. You can refereed following risk identification questionnaire.

5.2.2 Risk Analysis

The risks for the Project can be analyzed within the constraints of time and quality

ID	Risk Description	Probability	Impact
1	ML model integration with backend	Medium	High
2	Inconsistent dataset affecting accuracy	Medium	High
3	High computational requirements	Low	Medium
4	Delay in development process	Medium	High
5	Lack of real-time sensor data	High	Medium

Table 5.1: Short Risk Analysis Table

Probability	Value	Description
High	Probability of occurrence is	$> 75\%$
Medium	Probability of occurrence is	$26 - 75\%$
Low	Probability of occurrence is	$< 25\%$

Table 5.2: Risk Probability definitions

Impact	Value	Description
Very high	$> 10\%$	Schedule impact or Unacceptable quality
High	$5 - 10\%$	Schedule impact or Some parts of the project have low quality
Medium	$< 5\%$	Schedule impact or Barely noticeable degradation in quality Low Impact on schedule or Quality can be incorporated

Table 5.3: Risk Impact definitions

5.2.3 Overview of Risk Mitigation, Monitoring, Management

Following are the details for each risk.

Risk ID	1
Risk Description	Difficulty in integrating machine learning model (CNN-LSTM and XGBoost) with backend APIs for real-time prediction
Category	Technical / Development Environment
Source	System design and software requirement specification
Probability	Medium
Impact	High
Response	Mitigation
Strategy	Use simplified prediction model for real-time inference, modular API design, and incremental integration testing
Risk Status	Controlled

Table 5.4: Risk Detail Description

Risk ID	2
Risk Description	Incomplete or inconsistent dataset leading to reduced prediction accuracy in waterlogging detection
Category	Requirements / Data
Source	Dataset collection and preprocessing phase, based on software design and requirement specifications
Probability	Medium
Impact	High
Response	Mitigation
Strategy	Perform data preprocessing, handle missing values, apply normalization, and use simulated data to improve model training and reliability
Risk Status	Identified

Table 5.5: Risk Detail Description (Dataset Related)

Risk ID	3
Risk Description	High computational complexity and resource requirements of deep learning models (CNN-LSTM), which may affect real-time system performance
Category	Technology
Source	Identified during model development and testing phase while integrating deep learning models
Probability	Low
Impact	Very High
Response	Acceptance
Strategy	Use optimized or simplified prediction logic for real-time inference, while retaining the full hybrid model for offline analysis and validation
Risk Status	Identified

Table 5.6: Risk Detail Description (Technology Related)

5.3 Project Schedule

5.3.1 Project task set

Major Tasks in the Project stages are:

- Task 1: Requirement Analysis
 1. Identify problem statement
 2. Study existing systems
 3. Define project objectives
- Task 2: Design Phase
 1. Design system architecture
 2. Define data flow and modules
 3. Select technologies
- Task 3: Implementation Phase
 1. Develop machine learning model
 2. Implement backend APIs using FastAPI
 3. Develop frontend dashboard using React

- Task 4: Integration Phase
 1. Connect ML model with backend
 2. Integrate frontend with backend APIs
 3. Ensure smooth data flow
- Task 5: Testing Phase
 1. Test system functionality
 2. Validate prediction accuracy
 3. Fix bugs and improve performance
- Task 6: Deployment Phase
 1. Run system locally or on server
 2. Ensure system stability
- Task 7: Documentation Phase
 1. Prepare project report
 2. Document system design and results

5.3.2 Task network

Project tasks and their dependencies are represented in the following sequence diagram:

From Task	Action	To Task
Task 1: Requirements	Define Scope	Task 2: Literature
Task 2: Literature	Document Findings	Task 3: Design
Task 3: Design	Develop Model Design	Task 4: Model Dev
Task 4: Model Dev	Integrate AI + IoT	Task 5: Testing

Table 5.7: Task Network – Task Dependencies

5.3.3 Timeline Chart

Sr. No.	Project Phase / Task	Duration (Weeks)
1	Requirement Gathering & Problem Definition	2
2	Literature Survey & Feasibility Study	3
3	System Design & Architecture Development	2
4	Data Collection, Preprocessing & Model Training	4
5	System Integration, Testing & Documentation	3

Table 5.8: Project Timeline Chart

5.4 Team Organization

The manner in which the project team is organized and the mechanisms for reporting are outlined below. The team follows a collaborative and hierarchical structure to ensure smooth communication, accountability, and timely project execution.

5.4.1 Team Structure

The team structure for the project is defined to ensure effective collaboration and task distribution. Each member has been assigned a specific role based on their area of expertise to maintain efficiency and accountability.

Sr. No.	Role / Position	Responsibilities
1	Project Guide / Supervisor	Provides overall technical guidance, monitors progress, and ensures project objectives are achieved.
2	Team Leader	Oversees team coordination, schedules meetings, and ensures task completion as per project milestones.
3	Data Analyst	Collects, cleans, and preprocesses data from IoT sensors, GIS databases, and weather sources.
4	Machine Learning Engineer	Designs, trains, and evaluates predictive AI models for waterlogging detection and forecasting.
5	IoT & Integration Engineer	Develops IoT node connections, handles sensor deployment, and integrates data pipelines.
6	Documentation & Testing Lead	Maintains all technical documentation and ensures system validation and verification.

Table 5.9: Team Structure and Roles

5.4.2 Management Reporting and Communication

Mechanisms for project progress reporting and communication are structured to promote transparency and coordination among team members.

- **Weekly Review Meetings:** Conducted with the project guide to discuss progress, challenges, and deliverables.
- **Task Updates:** Each member provides status updates through a shared project tracker (Google Sheets or Trello).
- **Documentation:** Progress and technical findings are documented weekly in a shared drive.
- **Communication Channels:** Day-to-day coordination is maintained through group messaging (e.g., WhatsApp, Microsoft Teams).
- **Reporting Frequency:** Major milestones and interim results are reported biweekly to the supervisor as per the lab timetable.

6. Software requirement specification

6.1 Introduction

6.1.1 Purpose and Scope of Document

The purpose of this Software Requirement Specification (SRS) document is to provide a detailed description of the functional and non-functional requirements for the AI-driven Waterlogging Prediction System. It defines the project objectives, scope, system functionalities, performance criteria, and user expectations. The document serves as a reference for all stakeholders—including developers, supervisors, and evaluators—to ensure a clear understanding of the system design and implementation goals. It covers the entire lifecycle of the system, from data acquisition and preprocessing to AI model integration, prediction generation, and visualization through dashboards.

6.1.2 Overview of Responsibilities of Developer

The development team is responsible for designing, implementing, testing, and maintaining the proposed system. The key responsibilities include:

- Designing the system architecture and selecting appropriate technologies
- Collecting and preprocessing datasets related to rainfall, water levels, and drainage conditions
- Developing and training machine learning models (CNN-LSTM and XGBoost) for flood prediction
- Implementing backend services using FastAPI for data processing and API communication
- Developing the frontend dashboard using React for visualization and user

interaction

- Integrating machine learning models with backend APIs for real-time prediction
- Ensuring efficient communication between frontend and backend using REST APIs
- Testing the system for accuracy, performance, and reliability
- Debugging and optimizing system components
- Preparing documentation and maintaining version control using GitHub

6.2 Usage Scenario

This section describes how different users interact with the system and the various scenarios under which the system operates.

6.2.1 User Profiles

The system is designed for the following user categories:

- **Administrator**
 - Manages system configuration and monitors overall performance
 - Accesses prediction results and system data
- **Municipal Authority / Operator**
 - Uses the dashboard to monitor drainage conditions
 - Receives alerts and takes preventive actions
 - Analyzes risk levels and system recommendations
- **General User**
 - Views basic information about waterlogging risk
 - Accesses alerts and system updates

6.2.2 Use-cases

All use-cases for the software are presented. Description of all main Use cases using use case template is to be provided.

Sr No.	Use Case	Description	Actors	Assumptions
1	View Dashboard	User views real-time data such as water level, rainfall, and system indicators on the dashboard	User	System is active and data is available
2	Predict Risk	System processes input data using ML model to predict waterlogging risk levels	System	Input parameters (rainfall, water level) are available
3	Generate Alerts	System generates alerts when predicted risk exceeds threshold	System	Prediction model is functioning correctly
4	Chatbot Interaction	User interacts with chatbot to get information about drainage conditions	User	Chatbot service is active and API is working
5	Data Processing	Backend processes incoming data and sends results to frontend via API	System	Backend server and database are operational

Table 6.1: Use Case Table

6.2.3 Use Case View

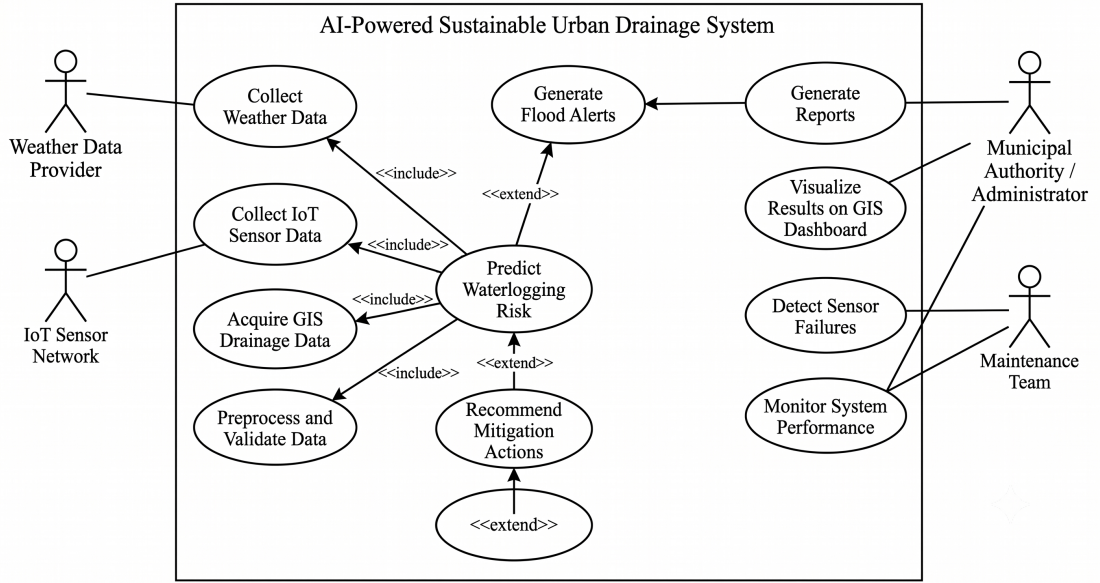


Figure 6.1: Use Case Diagram of Smart Drainage System

6.3 Data Model and Description

6.3.1 Data Description

The proposed system processes and manages multiple types of data related to urban drainage conditions and waterlogging prediction. The primary data used in the system includes environmental parameters such as rainfall intensity, water level, and drainage capacity.

These data inputs are either collected from datasets or simulated to represent real-world IoT sensor readings. The data is used for training and evaluating machine learning models as well as for real-time prediction through backend APIs.

The system handles data in different formats:

- **Structured Data:** Tabular data used for machine learning training and testing, typically stored in CSV format
- **Time-Series Data:** Sequential data representing rainfall and water level over time
- **JSON Data:** Used for communication between frontend and backend via REST APIs

The backend system processes input data, applies machine learning models, and returns prediction results such as flood risk levels. The processed data is visualized on the frontend dashboard for user interaction.

6.3.2 Data Objects and Relationships

The system consists of several data objects that represent different entities involved in the drainage monitoring and prediction process.

- **Sensor Data**
 - Attributes: rainfall, water level, drainage capacity, timestamp
 - Description: Represents environmental inputs used for prediction
- **Prediction Data**
 - Attributes: risk level (Low, Medium, High), probability, confidence
 - Description: Output generated by the machine learning model
- **User**
 - Attributes: user ID, role (Admin, Operator, User)
 - Description: Represents system users interacting with the dashboard
- **Alerts**
 - Attributes: alert ID, risk level, message, timestamp
 - Description: Generated when high-risk conditions are detected

Relationships:

- Sensor Data is processed by the Machine Learning Model to generate Prediction Data
- Prediction Data triggers Alerts when risk exceeds a threshold
- Users interact with Prediction Data and Alerts through the dashboard

The relationships between these data objects can be represented using an Entity-Relationship Diagram (ERD), where Sensor Data acts as the primary input entity, Prediction Data as the output entity, and Alerts and Users as supporting entities.

6.4 Functional Model and Description

The functional model describes the major operations of the proposed AI-based urban drainage system and the flow of data between different system components. The system follows a client-server architecture where the frontend interacts with the backend through REST APIs, and the backend processes data using machine learning models.

6.4.1 Data Flow Diagram

6.4.1.1 Level 0 Data Flow Diagram

The Level 0 Data Flow Diagram (DFD) represents the overall system as a single process interacting with external entities. The main entities include the User and the System. The user provides input data such as rainfall and water level, and the system processes this data to generate predictions and alerts.

Description for diagram: User → System → Prediction Output → Alerts → User

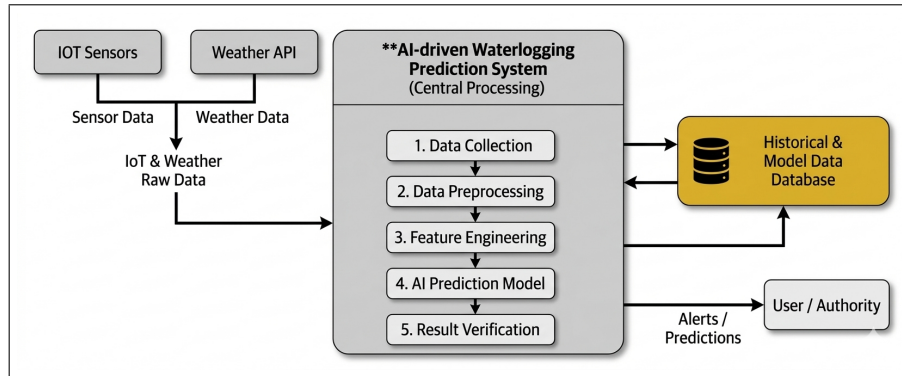


Figure 6.2: Level 0 Data Flow Diagram

6.4.1.2 Level 1 Data Flow Diagram

The Level 1 DFD provides a detailed breakdown of internal processes. The system is divided into multiple modules such as Data Input, Data Processing, Machine Learning Prediction, Alert Generation, and Visualization.

Description for diagram: Input Data → Data Processing → ML Model → Prediction → Alert Module → Dashboard

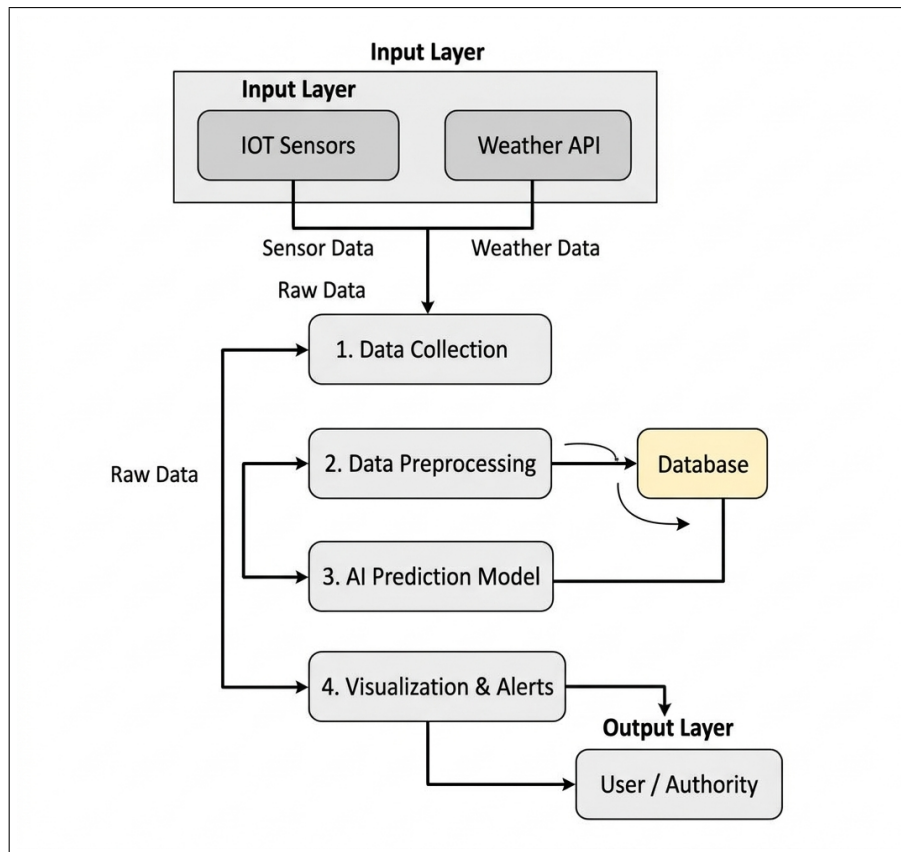


Figure 6.3: Level 1 Data Flow Diagram

6.4.2 Activity Diagram

- The Activity Diagram represents the sequence of operations performed by the system.
- It starts with user input or system-generated data.
- Data is processed and passed to the machine learning model.
- The model predicts the risk level.
- Based on the prediction, alerts are generated.
- Results are displayed on the dashboard.

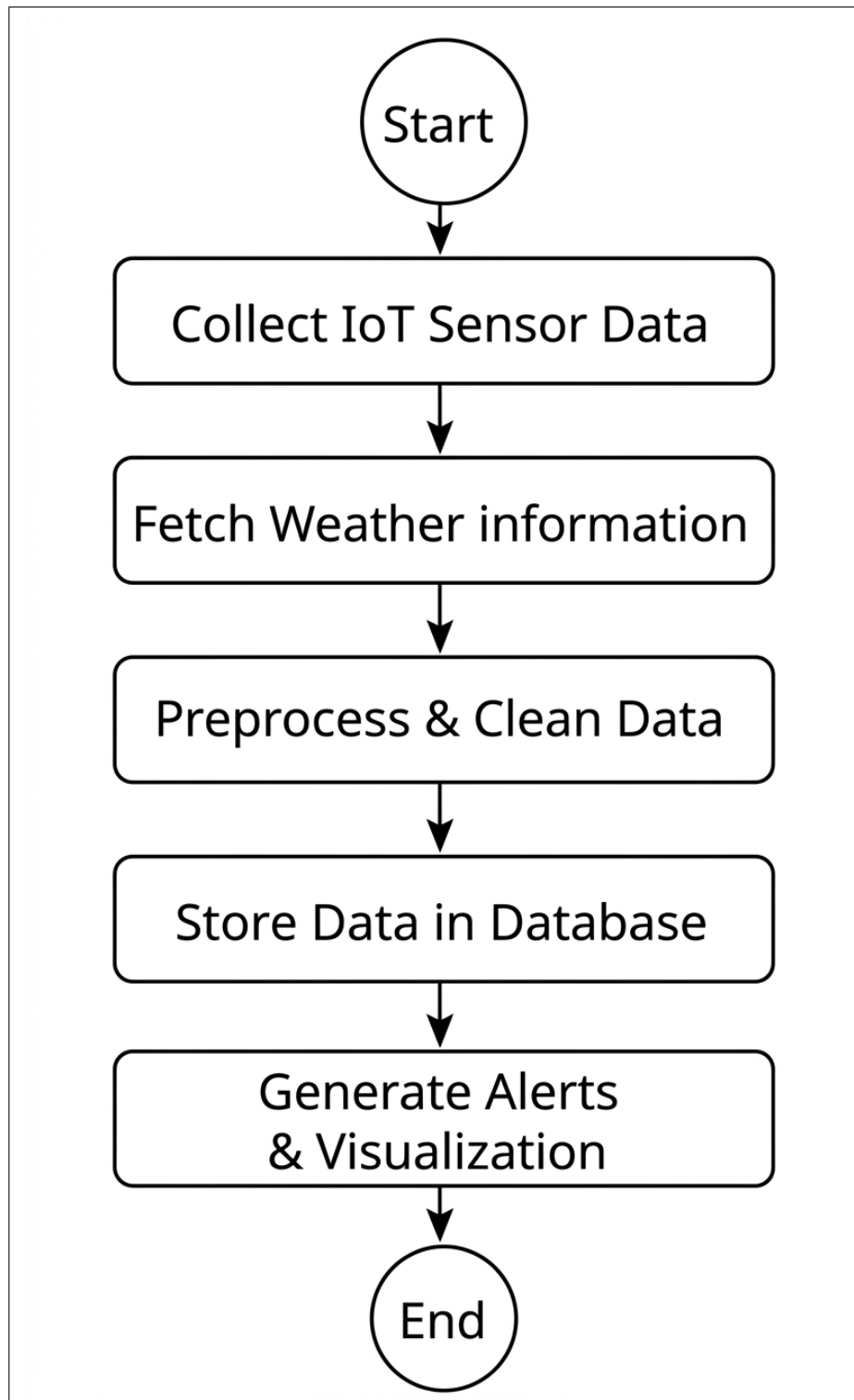


Figure 6.4: Activity Diagram

6.4.3 Non Functional Requirements

- **Interface Requirements**
 - User-friendly web interface developed using React

- Communication between frontend and backend using REST APIs (JSON format)
- **Performance Requirements**
 - System should provide real-time or near real-time prediction results
 - Backend APIs should handle multiple concurrent requests efficiently
- **Software Quality Attributes**
 - **Availability:** System should be accessible whenever required
 - **Modifiability:** System should support easy updates and enhancements
 - **Scalability:** System should handle increasing data and users
 - **Performance:** Fast response time for predictions and alerts
 - **Security:** Secure handling of data and API communication
 - **Testability:** Easy to test individual modules
 - **Usability:** Simple and intuitive dashboard for users

6.4.4 State Diagram

The state transition diagram represents different states of the system during operation. The system transitions between states such as Idle, Data Processing, Prediction, Alert Generation, and Display Output.

States:

- Idle State
- Data Input State
- Processing State
- Prediction State
- Alert State
- Display State

Transitions: The system moves from one state to another based on events such as receiving input data, completing prediction, or generating alerts.

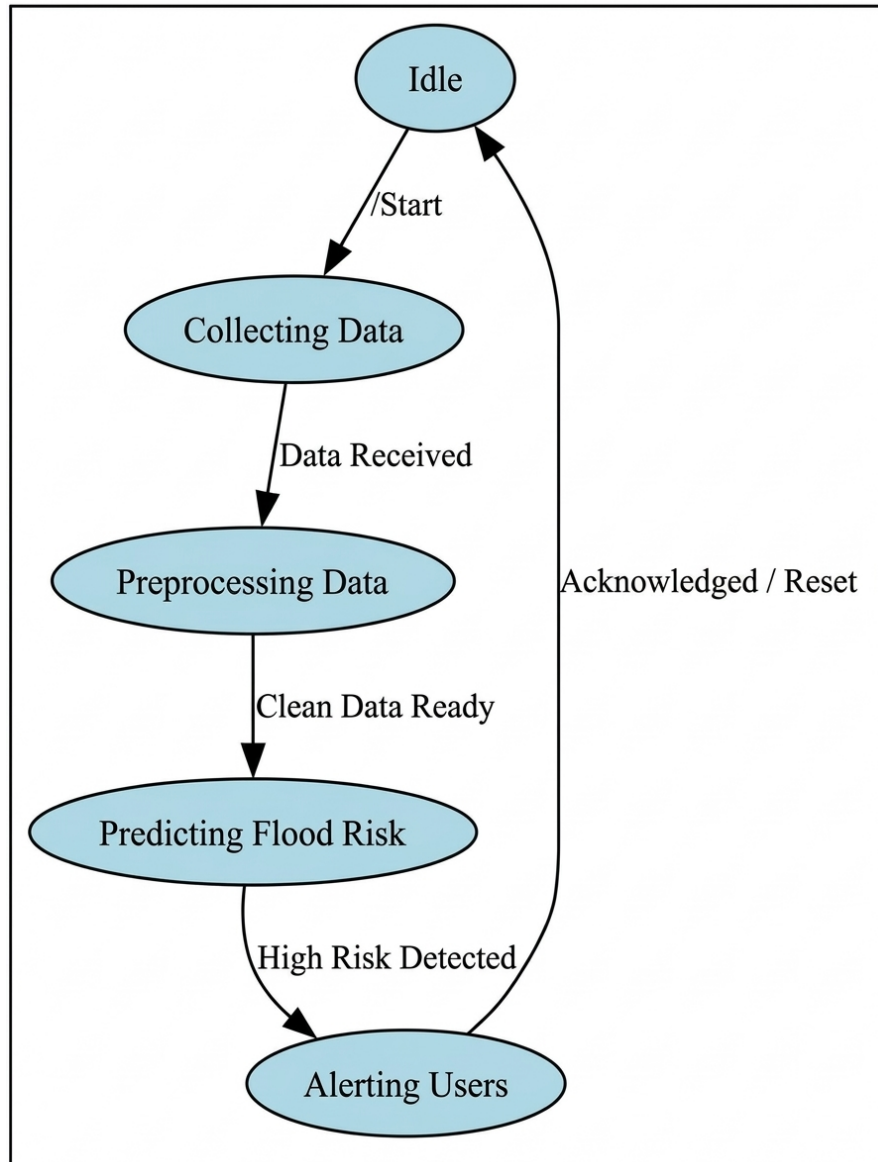


Figure 6.5: State Transition Diagram

6.4.5 Design Constraints

The system is subject to the following design constraints:

- Limited availability of real-time IoT sensor data (use of simulated dataset)
- Computational limitations for deploying complex deep learning models in real-time
- Dependency on internet connectivity for API communication
- Limited hardware resources for model execution

6.4.6 Software Interface Description

The system interacts with external components through well-defined interfaces:

- **Frontend Interface:** React-based dashboard for user interaction
- **Backend Interface:** FastAPI server handling data processing and prediction
- **Machine Learning Interface:** Integration of ML models for prediction
- **Database Interface:** Storage and retrieval of system data
- **API Interface:** REST APIs for communication between system components

7. Detailed Design Document using Appendix A and B

7.1 Introduction

This document provides a detailed design of the proposed system for "AI-based Sustainable Urban Drainage System for Waterlogging Prediction and Management." The design outlines how the system components interact to process input data, apply machine learning models, and generate predictions and alerts.

The system follows a modular and scalable architecture, where different components such as data processing, machine learning models, backend services, and frontend visualization are integrated to provide real-time monitoring and prediction capabilities.

7.2 Architectural Design

The system follows a layered client-server architecture consisting of frontend, backend, and machine learning components. Each module performs a specific function and communicates through well-defined interfaces.

Subsystems in the architecture include:

- **Data Input Layer:** Handles input data such as rainfall, water level, and drainage capacity from datasets or simulated sources
- **Processing Layer:** Performs data preprocessing and prepares input for machine learning models
- **Machine Learning Layer:** Implements prediction logic using CNN-LSTM and XGBoost models
- **Backend Layer:** Developed using FastAPI, handles API requests, model execution, and data communication

- **Frontend Layer:** Developed using React, provides dashboard visualization and user interaction

The architecture ensures separation of concerns, enabling scalability and maintainability. The backend processes data and sends prediction results to the frontend via REST APIs, where it is visualized for users.

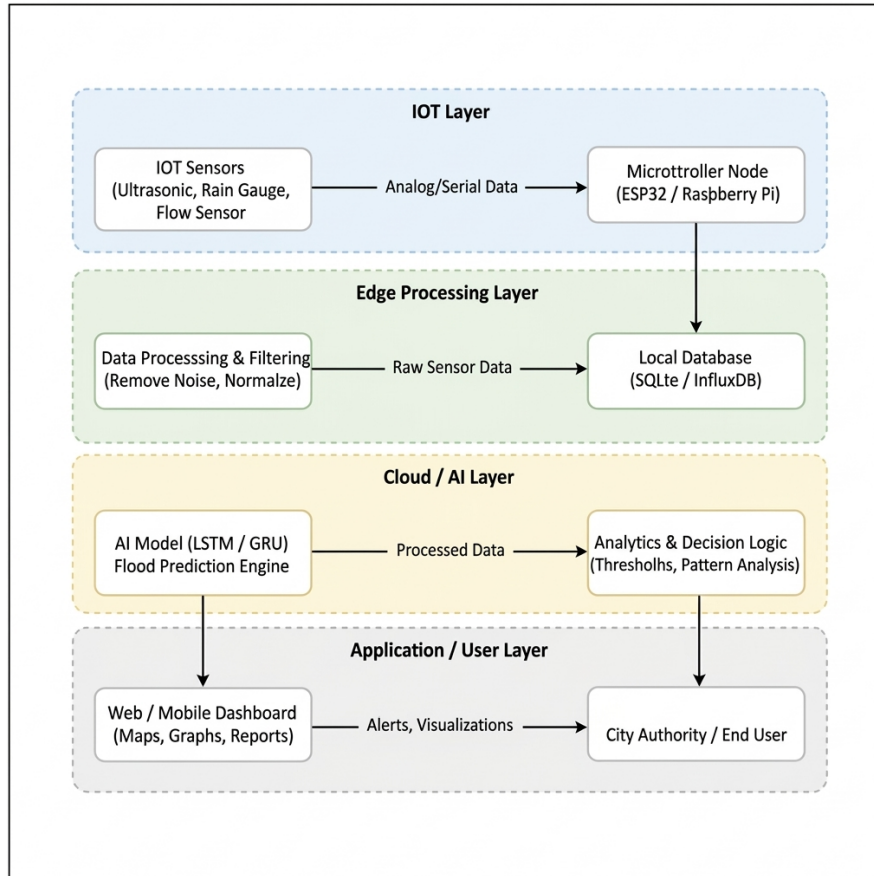


Figure 7.1: Architecture Diagram

7.3 Data Design (using Appendices A and B)

The system manages multiple types of data including input data, processed data, and prediction results. The data design ensures efficient handling, storage, and transmission of data between different system components.

7.3.1 Internal Software Data Structure

Internal data structures are used for communication between different components of the system. These include:

- Arrays for handling time-series data such as rainfall and water levels
- DataFrames for preprocessing and managing datasets during model training
- JSON objects for data exchange between frontend and backend

These structures enable efficient processing and transformation of data within the system.

7.3.2 Global Data Structure

Global data structures are accessible across multiple modules of the system. These include:

- Configuration parameters such as model paths and API settings
- Machine learning model instances loaded into memory
- Shared data variables used across backend services

These structures ensure consistency and efficient access to commonly used data..

7.3.3 Temporary Data Structure

Temporary data structures are used during intermediate processing stages. These include:

- Preprocessed input data before prediction
- Intermediate outputs from machine learning models
- Temporary API response data

These data structures are not permanently stored and are used only during runtime.

7.3.4 Database Description

The system uses a database to store application-related data such as user information, system logs, and prediction results.

The database includes the following entities:

- **User Table:** Stores user credentials and roles

- **Prediction Table:** Stores predicted risk levels and timestamps
- **Alert Table:** Stores generated alerts and related information

In addition to database storage, datasets are stored in file formats such as CSV for machine learning model training and evaluation.

7.4 Component Design

Class diagrams, Interaction Diagrams, Algorithms. Description of each component description required.

7.4.1 Class Diagram

The class diagram represents the object-oriented structure of the system, showing different classes, their attributes, methods, and relationships.

The major classes in the system include:

- **SensorData**
 - Attributes: rainfall, water_level, capacity
 - Methods: preprocess(), validate()
 - Description: Represents environmental input data used for prediction
- **PredictionModel**
 - Attributes: model parameters
 - Methods: loadModel(), predict()
 - Description: Implements machine learning models (CNN-LSTM and XGBoost) for risk prediction
- **AlertSystem**
 - Attributes: risk level, message
 - Methods: generateAlert()
 - Description: Generates alerts based on prediction results
- **APIService**
 - Attributes: endpoints
 - Methods: processRequest(), sendResponse()
 - Description: Handles communication between frontend and backend using REST APIs

- **UserInterface**

- Attributes: dashboard components
- Methods: displayData(), updateUI()
- Description: Represents frontend interface for user interaction

The relationships between classes include:

- Sensor Data is processed by Prediction Model
- Prediction Model generates output for Alert System
- APIService connects backend processing with User Interface

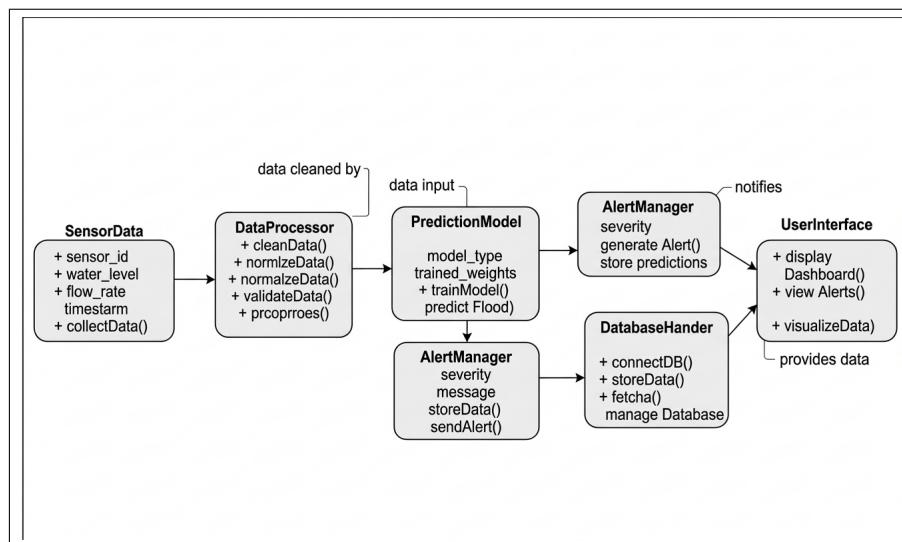


Figure 7.2: Class Diagram

7.4.2 Interaction Diagram

The interaction diagram describes how different components of the system communicate during execution.

Interaction Flow:

- User sends request through frontend dashboard
- API Service receives request in backend
- Data is processed and passed to Prediction Model
- Prediction Model generates risk output
- AlertSystem evaluates risk and generates alerts
- Results are sent back to frontend via API
- User views prediction and alerts on dashboard

7.4.3 Algorithms

Algorithm 1: Waterlogging Risk Prediction

1. Input rainfall, water level, and drainage capacity data
2. Pre-process input data
3. Pass data to machine learning model
4. Predict risk level (Low, Medium, High)
5. Return prediction result

Algorithm 2: Alert Generation

1. Receive predicted risk level
2. If risk is High, generate critical alert
3. Else if risk is Medium, generate warning alert
4. Else display normal status

8. Project Implementation

8.1 Introduction

This chapter describes the implementation details of the proposed AI-based urban drainage system for waterlogging prediction and management. The system integrates machine learning models with a web-based architecture consisting of a backend server and a frontend dashboard.

The implementation focuses on processing environmental data, applying machine learning algorithms for prediction, and displaying results through an interactive interface. The system is designed to support real-time prediction and alert generation using a modular approach.

8.2 Tools and Technologies Used

The system is developed using the following tools and technologies:

- **Programming Language:** Python, JavaScript
- **Frontend:** React.js for building interactive user interface and dashboard
- **Backend:** FastAPI for developing REST APIs and handling requests
- **Machine Learning:** TensorFlow (for CNN-LSTM model) and XGBoost for classification
- **Database:** MySQL for storing user data and system information
- **Libraries:** NumPy, Pandas, Scikit-learn for data processing
- **Version Control:** GitHub for code management and collaboration

8.3 Methodologies/Algorithm Details

The system uses a hybrid machine learning approach combining CNN-LSTM and XGBoost models for waterlogging prediction. The methodology involves the

following steps:

- Collect and preprocess input data such as rainfall, water level, and drainage capacity
- Use CNN-LSTM model to extract temporal features from time-series data
- Pass extracted features to XGBoost model for classification
- Generate prediction output as risk levels (Low, Medium, High)
- Use simplified prediction logic for real-time inference to improve performance

8.3.1 Algorithm 1: Waterlogging Prediction

Input: Rainfall, Water Level, Drainage Capacity

Output: Risk Level (Low, Medium, High)

1. Collect input data
2. Preprocess data (normalize, clean)
3. Convert data into time-series format
4. Pass data to CNN-LSTM model
5. Extract features from model
6. Scale features using trained scaler
7. Pass features to XGBoost model
8. Predict probability of risk
9. Classify risk level based on threshold
10. Return risk level

8.3.2 Algorithm 2: Alert Generation

Input: Risk Level

Output: Alert Message

1. Receive predicted risk level
2. If risk level is HIGH:
 - Generate critical alert

3. Else if risk level is MEDIUM:
 - Generate warning alert
4. Else:
 - Display normal status
5. Send alert to frontend dashboard

8.4 Verification and Validation for Acceptance

The system is verified and validated to ensure correctness, reliability, and performance.

Verification:

- Unit testing of backend APIs
- Validation of machine learning model outputs
- Checking data preprocessing steps

Validation:

- Testing system with sample datasets
- Evaluating prediction accuracy of ML models
- Verifying correct alert generation
- Ensuring proper frontend-backend communication

The system successfully meets the functional requirements and produces reliable prediction results, making it suitable for real-world applications in urban drainage management.

9. Software Testing

9.1 Type of Testing Used

To ensure the reliability and correctness of the system, different types of testing are performed during development.

- **Unit Testing**

- Individual modules such as API endpoints and machine learning functions are tested separately
- Ensures each component works correctly in isolation

- **Integration Testing**

- Interaction between frontend, backend, and machine learning modules is tested
- Ensures proper data flow between React frontend and FastAPI backend

- **System Testing**

- The complete system is tested as a whole
- Validates overall functionality including prediction, alerts, and dashboard visualization

- **Performance Testing**

- Tests system response time and API performance under multiple requests
- Ensures system can handle real-time prediction efficiently

9.2 Test Cases and Test Results

for each type of testing done.

9.2.1 Unit Testing

Test ID	Test Case	Expected Result	Status
UT1	API receives input data	Data is accepted and processed	Pass
UT2	ML model prediction function	Correct risk level is returned	Pass
UT3	Data preprocessing function	Data is normalized correctly	Pass

Table 9.1: Unit Testing

9.2.2 Integration Testing

Test ID	Test Case	Expected Result	Status
IT1	Frontend sends request to backend	Backend receives and processes data	Pass
IT2	Backend returns prediction to frontend	Prediction displayed correctly	Pass
IT3	Alert system integration	Alerts shown based on prediction	Pass

Table 9.2: Integration Testing

9.2.3 System Testing

Test ID	Test Case	Expected Result	Status
ST1	Complete system execution	System runs without errors	Pass
ST2	Prediction and alert generation	Correct alerts generated	Pass
ST3	Dashboard visualization	Data displayed correctly	Pass

Table 9.3: System Testing

10. Results

10.1 Screenshots

This section presents the graphical outputs of the implemented system. The screenshots demonstrate the functionality of the AI-based urban drainage system, including dashboard visualization, prediction results, alerts, and chatbot interaction.

- Dashboard displaying real-time water level and rainfall data
- Prediction results showing risk levels (Low, Medium, High)
- Alert system indicating critical conditions
- Chatbot interface for user interaction

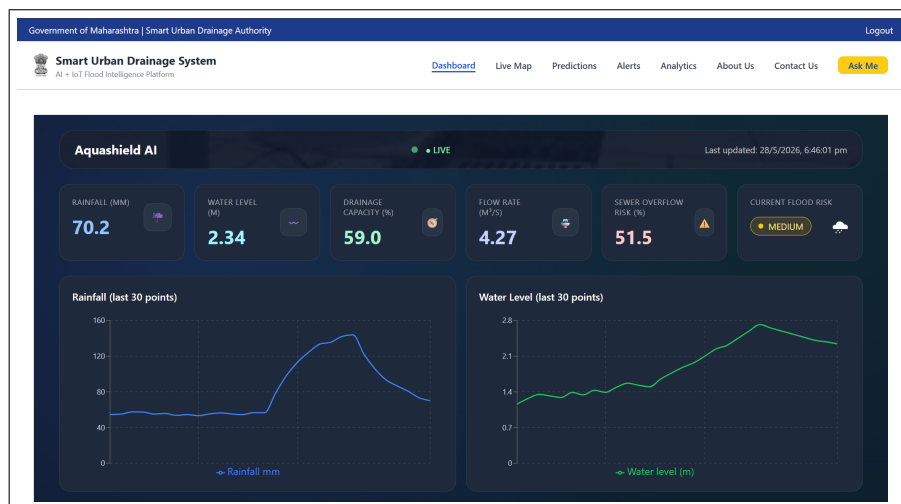


Figure 10.1: Dashboard



Figure 10.2: Graphs and Charts

Live Data Table (last 10 rows) Updated automatically every 30 seconds

Timestamp	Location	Rainfall	Water Level	Capacity	Flow	Overflow Risk	Temp	Humidity	Flood Risk
2026-05-28 18:42:01	Nigdi	135.7	2.45	63.4%	4.81	77.6%	33.1°C	88.2%	HIGH
2026-05-28 18:42:31	Bhosari	141.9	2.59	36.5%	4.70	73.5%	33.1°C	86.3%	HIGH
2026-05-28 18:43:01	Wakad	143.8	2.72	25.7%	5.13	83.3%	33.1°C	85.7%	HIGH
2026-05-28 18:43:31	Ravet	122.3	2.65	47.1%	4.89	69.0%	33.1°C	85.5%	HIGH
2026-05-28 18:44:01	Dehu Road	105.7	2.59	37.8%	4.66	65.8%	33.1°C	85.1%	HIGH
2026-05-28 18:44:31	Pimpri	93.5	2.53	51.3%	4.61	59.4%	33.2°C	84.1%	MEDIUM
2026-05-28 18:45:01	Chinchwad	86.7	2.47	90.7%	4.58	50.5%	33.2°C	83.4%	MEDIUM
2026-05-28 18:45:31	Akurd	80.8	2.41	77.0%	4.62	54.8%	33.2°C	83.0%	MEDIUM
2026-05-28 18:46:01	Nigdi	73.3	2.38	84.3%	4.33	43.9%	33.2°C	79.8%	MEDIUM
2026-05-28 18:46:01	Bhosari	70.2	2.34	59.0%	4.27	51.5%	33.2°C	80.0%	MEDIUM

Figure 10.3: Live Data Entry

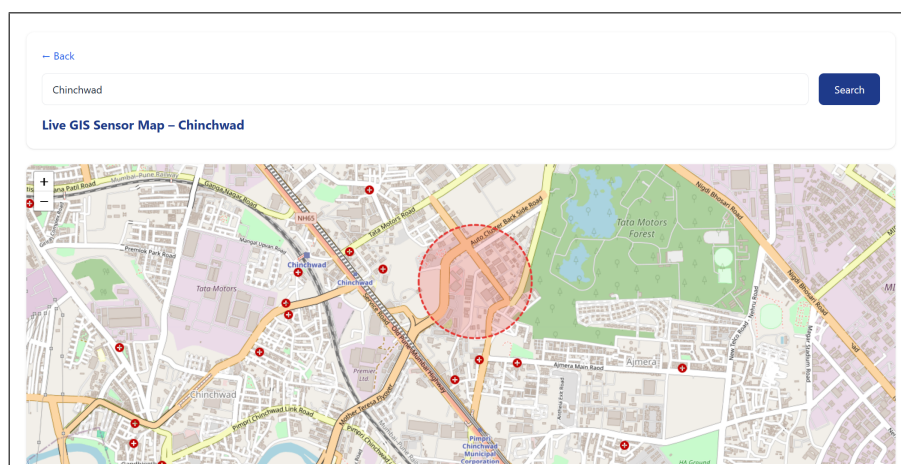


Figure 10.4: Searching map locations

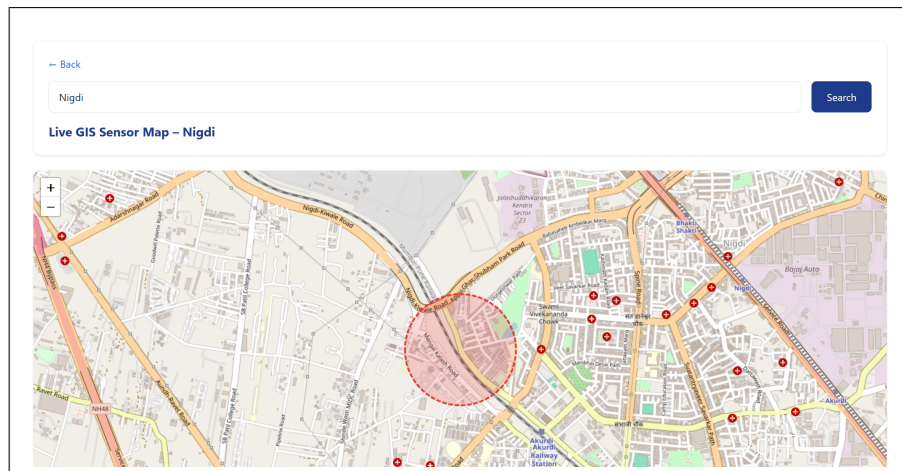


Figure 10.5: Searching map locations

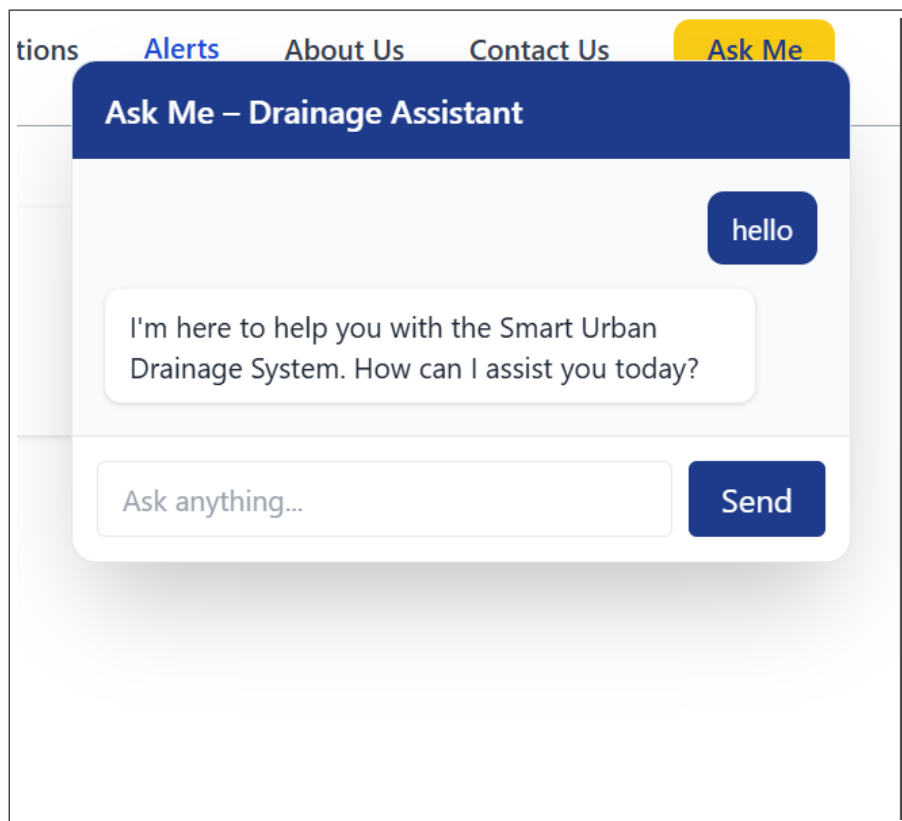


Figure 10.6: Chatbot interface for user interaction

10.2 Outputs

The system generates outputs in the form of predicted risk levels and corresponding alerts. These outputs are derived from the machine learning model based on input parameters such as rainfall, water level, and drainage capacity.

11. Deployment and Maintenance

11.1 Installation and Un-installation

The system is deployed as a web-based application consisting of a backend server and a frontend interface. The following steps describe the installation process:

Installation Steps:

1. Clone the project repository from GitHub
2. Navigate to the project directory
3. **Backend Setup:**
 - Create and activate a virtual environment
 - Install required dependencies using: `pip install -r requirements.txt`
 - Configure environment variables (e.g., API keys, database URL)
 - Start the backend server using:
`uvicorn app.main:app --reload`
4. **Frontend Setup:**
 - Navigate to frontend directory
 - Install dependencies:
`npm install`
 - Run the frontend application:
`npm run dev`
5. Access the system through the browser using the provided local URL

Un-installation Steps:

- Stop backend and frontend servers
- Delete the project directory
- Remove virtual environment and installed dependencies

11.2 User Help

The system provides an intuitive interface that allows users to easily interact with the application.

Basic Usage Instructions:

- Open the dashboard through the web browser
- View real-time system indicators such as rainfall, water level, and drainage status
- Observe predicted risk levels displayed on the dashboard
- Monitor alerts generated for critical conditions
- Use the chatbot feature to get additional information and assistance

Help Features:

- User-friendly interface with clear visualization
- Dashboard indicators for quick understanding of system status
- Chatbot support for answering user queries
- Error messages and notifications for invalid inputs

Maintenance Guidelines:

- Regularly update datasets and machine learning models
- Monitor system performance and API response time
- Ensure proper functioning of backend services
- Update dependencies and libraries when required

12. Conclusion and Future Scope

12.1 Summary

The project "AI for Sustainable Urban Drainage System for Effective Waterlogging Prediction and Management" focuses on developing an intelligent system to predict and manage urban waterlogging conditions. The system integrates machine learning models, backend services, and a user-friendly frontend dashboard to provide real-time monitoring and prediction. The implementation involves processing environmental parameters such as rainfall, water level, and drainage capacity. A hybrid machine learning approach combining CNN-LSTM and XGBoost is used to analyze temporal patterns and classify flood risk levels. The system is deployed using a FastAPI backend and a React-based frontend, enabling efficient data communication and visualization.

12.2 Conclusion

The proposed system successfully demonstrates the use of artificial intelligence in urban drainage management. It provides accurate prediction of waterlogging risk levels and generates alerts for critical situations. The system is capable of handling real-time data inputs and presenting results through an interactive dashboard. The integration of machine learning with web technologies ensures scalability, efficiency, and usability. The project highlights the potential of AI-driven solutions in improving decision-making and reducing the impact of urban flooding.

12.3 Future Scope

The system can be further enhanced in several ways:

- Integration with real-time IoT sensors for accurate and live data collection

- Incorporation of GIS-based mapping for location-specific predictions
- Deployment on cloud platforms for better scalability and accessibility
- Improvement of machine learning models for higher prediction accuracy
- Development of a mobile application for wider accessibility
- Implementation of automated drainage control systems based on predictions
- Integration with weather forecasting APIs for advanced prediction capabilities

Annexure A

References

1. D. Misra, G. Das, and D. Das, “Artificial intelligent based smart drainage system,” 2022, preprint available at Research Square. [Online]. Available: <https://doi.org/10.21203/rs.3.rs-1338435/v1>
2. R. Dabas, T. Imam, F. Safwat, S. Rizwan, and K. Alam, “Smart drainage system for urban flood prevention.”
3. M. M. Saddiqi, W. Zhao, S. Cotterill, and R. K. Dereli, “Smart management of combined sewer overflows: From an ancient technology to artificial intelligence,” *Wiley Interdisciplinary Reviews: Water*, vol. 10, no. 3, p. e1635, 2023. [Online]. Available: <https://doi.org/10.1002/wat2.1635>
4. S. Boughandjioua, F. Laouacheria, and N. Azizi, “Integrating artificial intelligence for urban drainage systems aided decision: State of the art,” *International Journal of Artificial Intelligence: Theory and Applications*, vol. 2, no. 2, pp. 1–9, 2023. [Online]. Available: <https://int-j-aita.univ-annaba.dz/wp-content/themes/cbusiness-investment/view/vol2iss2/vol2iss2-1.pdf>
5. D. P. Selvam, G. G. Reddy, and N. M. Kumar, “Designing a smart and safe drainage system using artificial intelligence,” *International Journal of Engineering Research and Technology (IJERT)*, vol. 9, pp. 497–500, 2020. [Online]. Available: <https://www.ijert.org/research/designing-a-smart-and-safe-drainage-system-using-artificial-intelligence-IJERT.pdf>
6. K. Haripriya, S. S. Hammed, C. Preethi, S. Pavalarajan, R. Padmakumar, and M. M. Surya, “Automated drainage management system: AI and IoT-based blockage detection for smarter cities,” in *7th International Conference on Inventive Material Science and Applications (ICIMA)*, IEEE, 2025, pp.

- 998–1004. [Online]. Available: <https://doi.org/10.1109/ICIMA64861.2025.11073840>
7. S. H. Kwon and J. H. Kim, “Machine learning and urban drainage systems: State-of-the-art review,” *Water*, vol. 13, no. 24, p. 3545, 2021. [Online]. Available: <https://doi.org/10.3390/w13243545>
 8. P. S. Thanigaivelu, A. Sairam, P. Vijayan, A. S. Kumar, N. Mohankumar, and T. R. GaneshBabu, “Revolutionizing urban drainage: A smart IoT approach to stormwater management using adaboosting algorithm,” in *International Conference on Advances in Modern Age Technologies for Health and Engineering Science (AMATHE)*, IEEE, 2024, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/AMATHE61652.2024.10582217>
 9. R. Sekita, T. Watanabe, T. Yoshizawa, and K. Nakanishi, “Reliability improvement of rainwater-drainage system using IoT and AI,” in *Annual Reliability and Maintainability Symposium (RAMS)*, IEEE, 2024, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/RAMS51492.2024.10457666>
 10. S. Veerappan, “Edge-enabled smart storm water drainage systems: A real-time analytics framework for urban flood management,” *Journal of Smart Infrastructure and Environmental Sustainability*, vol. 1, no. 1, pp. 52–59, 2024. [Online]. Available: <https://doi.org/10.17051/JSIES/01.01.06>
 11. U. K. Sharma and A. Maurya, “AI for predictive urban water drainage systems,” *International Journal of Progressive Research in Engineering Management and Science*, vol. 4, no. 12, pp. 763–767, 2024. [Online]. Available: <https://www.doi.org/10.58257/IJPREMS37638>
 12. P.-C. Huang and K. T. Lee, “An alternative for predicting real-time water levels of urban drainage systems,” *Journal of Environmental Management*, vol. 347, p. 119099, 2023. [Online]. Available: <https://doi.org/10.1016/j.jenvman.2023.119099>
 13. Y. Liu, Y. Liu, J. Zheng, F. Chai, and H. Ren, “Intelligent prediction method for waterlogging risk based on AI and numerical model,” *Water*, vol. 14, no. 15, p. 2282, 2022. [Online]. Available: <https://doi.org/10.3390/w14152282>

Annexure B

Laboratory Assignments on Project Analysis of Algorithmic Design

- **Problem Definition and Feasibility using Knowledge Canvas and IDEA Matrix**

The problem addressed in this project is the prediction and management of urban waterlogging using artificial intelligence techniques. The increasing frequency of urban flooding due to inadequate drainage systems highlights the need for an intelligent monitoring and prediction system.

The feasibility of the proposed system is analyzed using the Knowledge Canvas and IDEA Matrix concepts . The Knowledge Canvas helps in identifying opportunities by analyzing problem space, available technologies, and user needs. The IDEA Matrix is used to evaluate innovation strategies for improving the solution.

IDEA Matrix Interpretation for the Project:

- **Increase:** Enhance prediction accuracy using hybrid machine learning models
- **Drive:** Promote smart city solutions for urban drainage management
- **Educate:** Inform users and authorities about flood risks
- **Accelerate:** Enable faster decision-making through real-time predictions
- **Improve:** Improve system performance and user interface
- **Deliver:** Provide reliable prediction and alert system
- **Evaluate:** Continuously assess model performance
- **Associate:** Integrate with external data sources such as weather APIs
- **Ignore:** Avoid unnecessary system complexity

- **Decrease:** Reduce prediction latency
- **Eliminate:** Remove redundant data processing steps
- **Avoid:** Avoid overfitting in machine learning models

The project is feasible from a technical and practical perspective as it uses available datasets, machine learning techniques, and web technologies to provide a scalable solution.

- **Feasibility Assessment using NP-Hard / NP-Complete Concepts**

The waterlogging prediction problem involves analyzing multiple parameters such as rainfall, water level, and drainage capacity. While the problem itself is not strictly NP-Hard, it involves complex decision-making due to multiple input variables and temporal dependencies.

The prediction function can be represented as:

$$y = f(x)$$

where:

- x represents input variables such as rainfall, water level, and drainage capacity
- y represents the predicted risk level (Low, Medium, High)

Machine learning models are used to approximate the function $f(x)$ efficiently, avoiding exhaustive computation. This makes the problem computationally feasible for real-time applications.

—

- **Input-Output Relationship**

The system processes input data and generates corresponding outputs as follows:

- **Input (x):**
 - * Rainfall intensity
 - * Water level
 - * Drainage capacity
- **Output (y):**
 - * Predicted waterlogging risk level (Low, Medium, High)
 - * Alert messages based on risk level

Thus, the system models a function $y = f(x)$, where machine learning algorithms are used to learn the mapping between environmental conditions and flood risk levels.

Annexure C

Laboratory Assignments on Project Quality and Reliability Testing of Project Design

- **Divide and Conquer Strategy and Functional Analysis**

The proposed system follows a divide and conquer strategy by decomposing the overall problem into smaller, manageable modules such as data input, preprocessing, machine learning prediction, alert generation, and visualization.

Each module operates independently and communicates through well-defined interfaces, enabling concurrent and distributed processing. For example, data preprocessing and prediction modules can operate in parallel to improve system efficiency.

From a mathematical perspective:

- **Objects:** Sensor Data, Prediction Model, Alert System, User Interface
- **Morphisms:** Data transformation functions such as preprocessing and prediction
- **Functional Relations:** Mapping of input data to output prediction
 $y = f(x)$
- **Overloading:** Different API endpoints handling various input formats

Logical relationships between inputs and outputs can be represented using function relations and state transitions. For instance, increasing rainfall and water level leads to a higher risk state.

- **Functional Dependency and Software Modeling**

The system components are interconnected through functional dependencies. The dependency graph is as follows:

Input Data $\rightarrow$ Processing $\rightarrow$ ML Model $\rightarrow$ Prediction $\rightarrow$ Alerts $\rightarrow$

Dashboard

Software modeling techniques such as UML diagrams are used to represent system behavior and structure:

- Use Case Diagram: Represents interaction between users and system
- Class Diagram: Defines system structure and object relationships
- Activity Diagram: Shows workflow of system operations
- State Diagram: Represents system states and transitions

These models help in understanding system dependencies and improving design reliability.

- **Testing Using Generated Data and Reliability Analysis**

The system is tested using generated datasets and sample inputs to evaluate its correctness and reliability. Mathematical models are used to simulate input conditions such as rainfall intensity and water levels.

Testing Approach:

- Generate test data representing different environmental conditions
- Apply inputs to machine learning model
- Validate predicted outputs against expected behavior
- Verify alert generation based on risk levels

Black Box Testing (Sample Test Cases):

Test ID	Input	Expected Output	Status
1	Low rainfall, low water level	Low risk prediction	Pass
2	Moderate rainfall, medium water level	Medium risk prediction	Pass
3	High rainfall, high water level	High risk prediction and alert	Pass
4	Invalid input data	Error message or rejection	Pass

Table C.1: Black Box Testing

Testing tools such as Python-based frameworks and API testing tools are used to validate system functionality. UML diagrams are also verified to ensure consistency with system behavior.

- **Additional Assignments**

Additional analysis includes evaluating system reliability, scalability, and performance under different conditions. The system demonstrates high reliability by providing consistent prediction outputs and handling user interactions efficiently.

Annexure D

Project Planner

The project planning is carried out using standard project management techniques to ensure systematic development and timely completion of the proposed system. A planner tool (such as Gantt chart or project scheduling tool) is used to define tasks, timelines, and dependencies among different phases.

The project is divided into multiple phases based on the Waterfall model, where each phase is completed sequentially. The major activities and their scheduling are outlined below:

- **Phase 1: Problem Definition and Literature Survey**
 - Understanding the problem of urban waterlogging
 - Reviewing existing research and solutions
- **Phase 2: Dataset Collection and Preprocessing**
 - Collecting environmental data (rainfall, water level, etc.)
 - Cleaning and preparing data for model training
- **Phase 3: Machine Learning Model Development**
 - Developing CNN-LSTM and XGBoost models
 - Training and validating the models
- **Phase 4: Backend Development**
 - Implementing APIs using FastAPI
 - Integrating machine learning models
- **Phase 5: Frontend Development**
 - Designing dashboard using React
 - Implementing visualization components
- **Phase 6: System Integration**
 - Connecting frontend, backend, and ML modules

- Ensuring smooth data flow
- **Phase 7: Testing and Validation**
 - Performing unit, integration, and system testing
 - Verifying system accuracy and performance
- **Phase 8: Documentation and Report Preparation**
 - Preparing project report and documentation

The detailed project schedule, including task dependencies and timelines, is represented using a project planner (Gantt chart) and is included in Annex C.

Annexure E

Published Papers

1. **Paper Title:** A Smart Computing Framework for AI-Driven Urban Drainage and Waterlogging Prediction
2. **Status:** Under review
3. **Target Conference/Journal:** 2nd International Conference on Intelligent Computing and Sustainable Innovation in Technology (IC-SIT) 2026, ICNIEEE-2026

Annexure F

Plagiarism Report

The project report has been checked for plagiarism using a standard plagiarism detection tool. The similarity index of the document is within the acceptable limits as per institutional guidelines. The report has been carefully written by paraphrasing all referenced materials and properly citing external sources. All figures, tables, and content used from external references have been acknowledged appropriately.

The plagiarism report is attached below for verification.

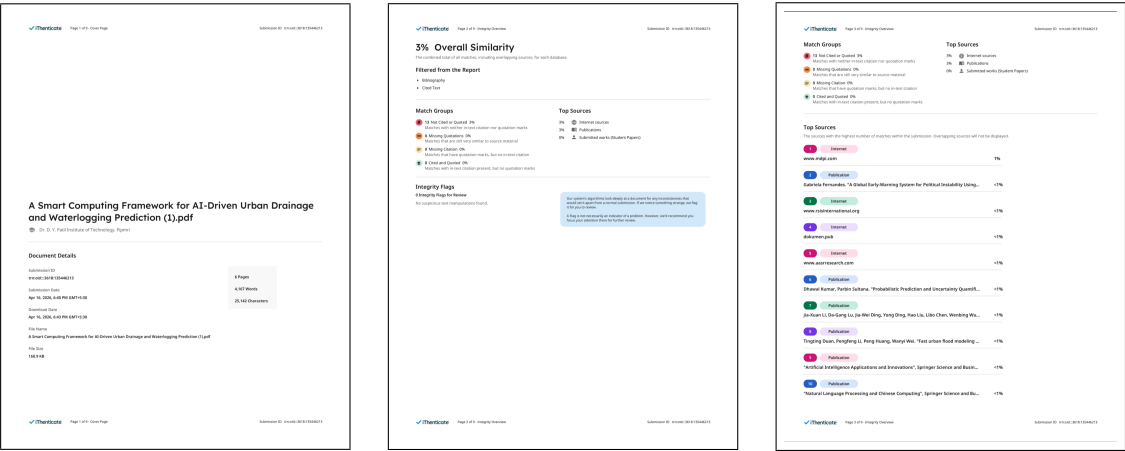


Figure F.1: Plagiarism Report 1

Figure F.2: Plagiarism Report 2

Figure F.3: Plagiarism Report 3

Annexure G

Term-II Project Laboratory Assignments

1. Review of Design and Corrective Actions

Based on the feedback received during Term-I assessment, several improvements were made to the system design and implementation. The initial prototype was enhanced by improving model integration, refining user interface components, and optimizing backend performance.

Corrections included:

- Improved data preprocessing techniques for better prediction accuracy
- Enhanced integration between machine learning model and backend APIs
- Improved dashboard visualization and user experience
- Addition of chatbot functionality for better user interaction

Participation in project discussions and technical reviews helped in refining the system design and improving overall system reliability.

—

2. Project Workstation Selection and Setup

A suitable development environment was selected to implement the system. The setup includes:

- Operating System: Windows/Linux
- Programming Environment: Python and Node.js
- Backend Setup: FastAPI with required dependencies
- Frontend Setup: React.js development environment
- Machine Learning Libraries: TensorFlow, XGBoost, Scikit-learn

All required tools and libraries were installed and configured successfully. The project environment was verified to ensure smooth execution of both backend and frontend modules.

3. Programming and Implementation

The system was developed by implementing various modules as per the design defined in Term-I, along with necessary improvements.

The implementation includes:

- Machine learning model development for waterlogging prediction
- Backend API development using FastAPI
- Frontend dashboard development using React
- Integration of ML model with backend services
- Implementation of alert system and chatbot functionality

All modules were integrated to form a complete working system capable of real-time prediction and visualization.

4. Testing and Reliability Analysis

Appropriate testing tools and techniques were used to validate system performance and reliability.

- Unit testing for backend APIs and ML functions
- Integration testing for frontend-backend communication
- System testing for overall functionality
- Reliability testing using different input scenarios

Various test cases were executed, and results were analyzed using tables and graphs. The system demonstrated consistent performance and accurate prediction results under different conditions.

Additional Assignment for Entrepreneurship Project:

5. Installation and Reliability Testing at Client End

The system can be deployed in a real-world environment such as municipal offices or smart city platforms. Installation procedures were tested to ensure easy deployment.

Reliability testing includes:

- System performance under continuous usage
- Stability of backend APIs
- Accuracy of prediction model over time

The system shows potential for real-world deployment with further enhancements such as IoT integration and cloud-based scalability.

Annexure H

Information of Project Group Members



1. Name : Tushar Ramsing Chavhan
2. Date of Birth : 15/08/2004
3. Gender : Male
4. Permanent Address : Mauli Niwas, Mohone, Kalyan
5. E-Mail : chavhantushar223@gmail.com
6. Mobile/Contact No. : 7738184213
7. Placement Details : Infosys
8. Paper Published : Accepted



1. Name : Sakshi Ingale
2. Date of Birth : 22/12/2003
3. Gender : Female
4. Permanent Address : Near Jain Temple, Dattawad, Tal-Shirol, Dist-Kolhapur
5. E-Mail : sakshiingale2212@gmail.com
6. Mobile/Contact No. : 7276902003
7. Placement Details : Cognizant
8. Paper Published : Accepted



1. Name : Vrushali Dhage
2. Date of Birth : 19/09/2004
3. Gender : Female
4. Permanent Address : At Post Chambhurdi, Tal- Shrigonda, Dist- Ahilyanagar
5. E-Mail : vrushalidhage19@gmail.com
6. Mobile/Contact No. : 8010332788
7. Placement Details : -
8. Paper Published : Accepted



1. Name : Snehal Dnyaneshwar Godage
2. Date of Birth : 08/10/2004
3. Gender : Female
4. Permanent Address : At-Sogaon Post-Kedgaon, Tal-Karamala, Dist- Solapur,
Pin-413202
5. E-Mail : sgodage46@gmail.com
6. Mobile/Contact No. : 8262925894
7. Placement Details : Cognizant
8. Paper Published : Accepted